



AWS Session

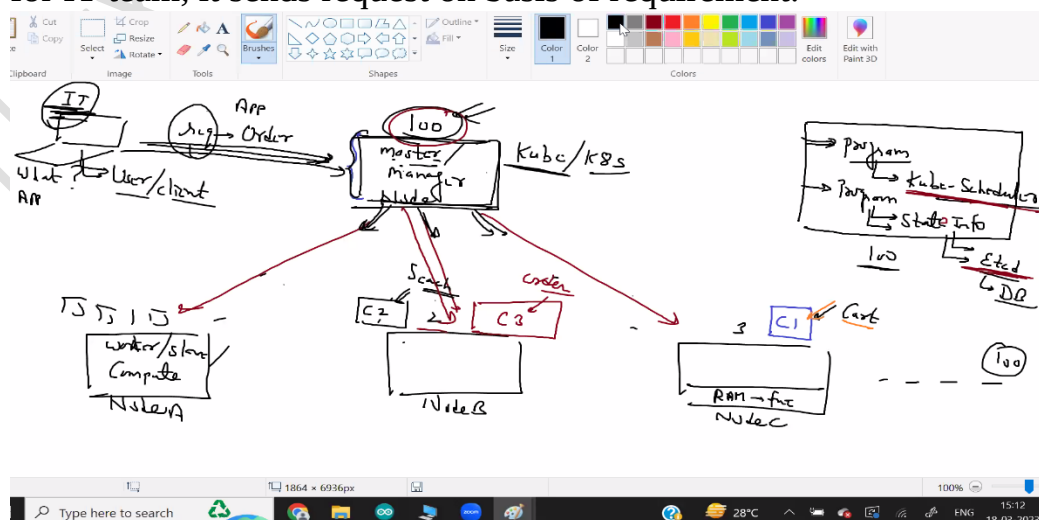
18-3-2023

- Various IT Companies are using EKS- Elastic Kubernetes Service of AWS. Companies like Goldman Sach, Netflix, Apple, Amazon Ecommerce website and many more..
- **EKS** is a managed service of AWS. It is one service of AWS that have a huge potential to perform managed service of kubernetes. Entire management of k8s cluster is managed by EKS service of Aws.
- Here k8s cluster includes storage, LB, active directories, RAM/CPU and lot more..Powerful thing about EKS is that it is tightly integrated with AWS other services like ec2 etc..
- **eksctl Tool**- It is a 3rd party tool to manage EKS.
- If we want to setup webapp we need physical Hardware like RAM/CPU, also we need Operating System and a runtime to run the webserver. So there are 3 possibilities to setup this-
 1. **Baremetal**- Running directly on the laptop, Here we use OS
 2. **Virtualization**- Launch a virtual machine or in AWS we use instance and we have to configure the runtime there. Here we use Virtual machine.
 3. **Containerization**- Launch VM, install, start(boot) and run services in less then a second. Here we use containers.

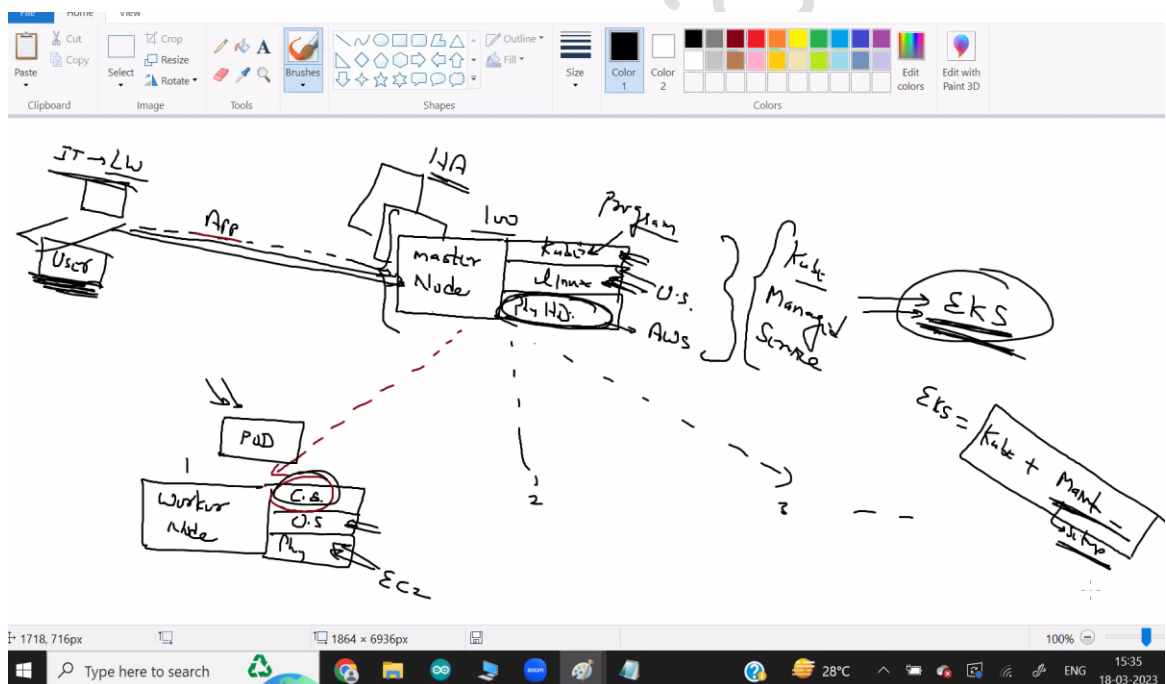
So, OS=VM=Container

- Today's Requirement is we need agility, speed and great performance. In container world if something fails it launches a new within a second so no downtime.
- To launch Container we have **ECS- Elastic Container service** in AWS.
- Launching container is not we want we want when something fails it launch a new container to make the application run.
- We have Container Engine that helps to run the containers. Some of the examples are CRI-O, containerd etc..Docker use containerd.
This program is known as **Hypervisor**.

- Where the container engine runs it is known as **host OS** or **container OS** also called **Compute Node/Container Node**.
Reason why compute node, because it is the one who gives RAM/CPU etc.
- EKS uses container engine called **containerd**.
- Challenge is if we want to launch 1000 containers or nodes then management of these nodes will be harder. So due to any reason if one container fails then application will also fail. So, we want a program who monitors everything, we need automation. Duty of that program is to keep on monitoring all containers as soon as if one container goes down it will up the other. This kind of monitoring is called **Realtime monitoring**. If we want our program to be fault tolerant then we need this program that manages container so for that we have container management tool called **Kubernetes**. Other examples are docker, swarm etc.
- In Kubernetes main system is called **Manager computer or manager node**. Any computer that is running everything of our application we refer that as the **worker node/slave node**.
- Container which is available and we have a requirement so the one who takes decision to launch is the **kubescheduler**.
- This entire setup is the **Container cluster**.
- Storing information of cluster also called **state information** so, to manage we have **etcd(kind of database to store information)**.
- One of the core facilities EKS gives is **Fault Tolerance and Seamless facility** (means as soon as 1 system goes down it automatically connect, seamlessly to different system, no downtime). Program that gives seamless facility is called **Kubeproxy**.
- In master node one more program is running called **kubeAPI** that is meant for IT team, it sends request on basis of requirement.



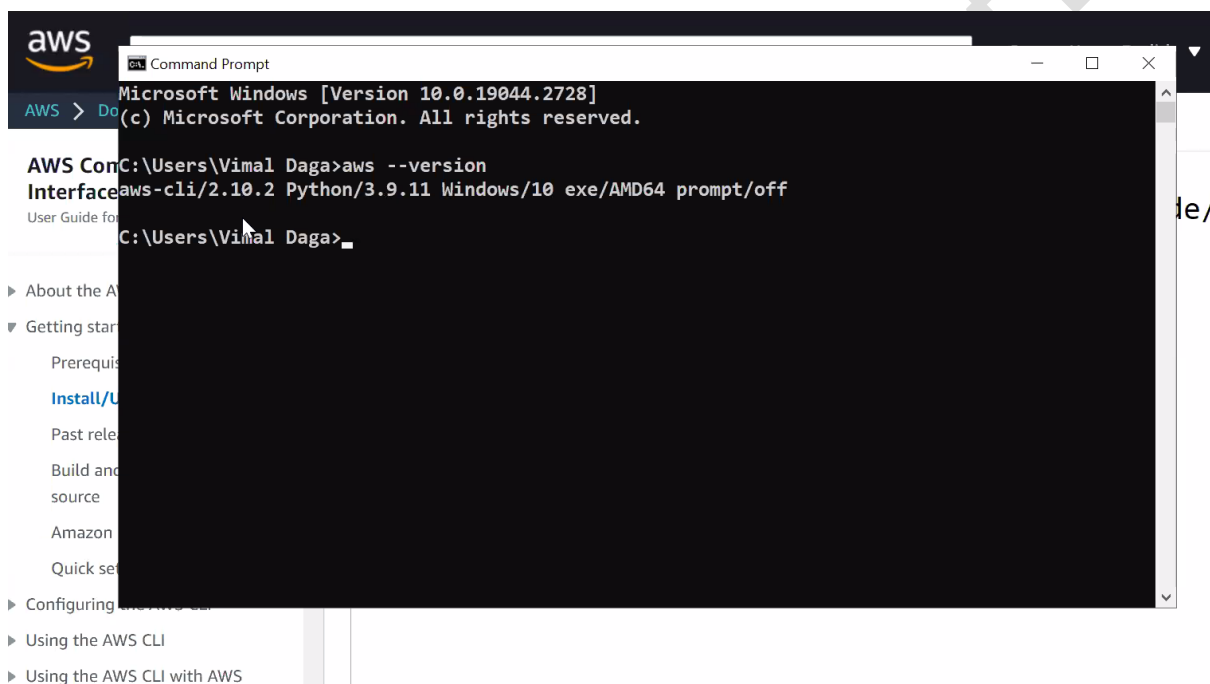
- In k8s container is known as a **POD**.
So, Pod==container== OS(RAM/CPU/HD)
- Challenge- If we want to launch anything we need physical Hardware- RAM/CPU/HD so we have to purchase on demand it will cost a huge amount. So to solve this challenge we have EKS service of AWS. It is a managed service; everything will be managed by AWS. So EKS is a k8s service but is managed by AWS.
- **Single Point of failure**- When master fails entire cluster will fail. So, to solve single point of failure challenge instead of 1 master we can launch more master nodes. Master to have multiple copies called **high availability**. So, this high availability will also be setup by AWS.
- **EKS**→ Entire k8s+management/maintenance of k8s
Means setting up with all best practices like security, reliability, availability etc.



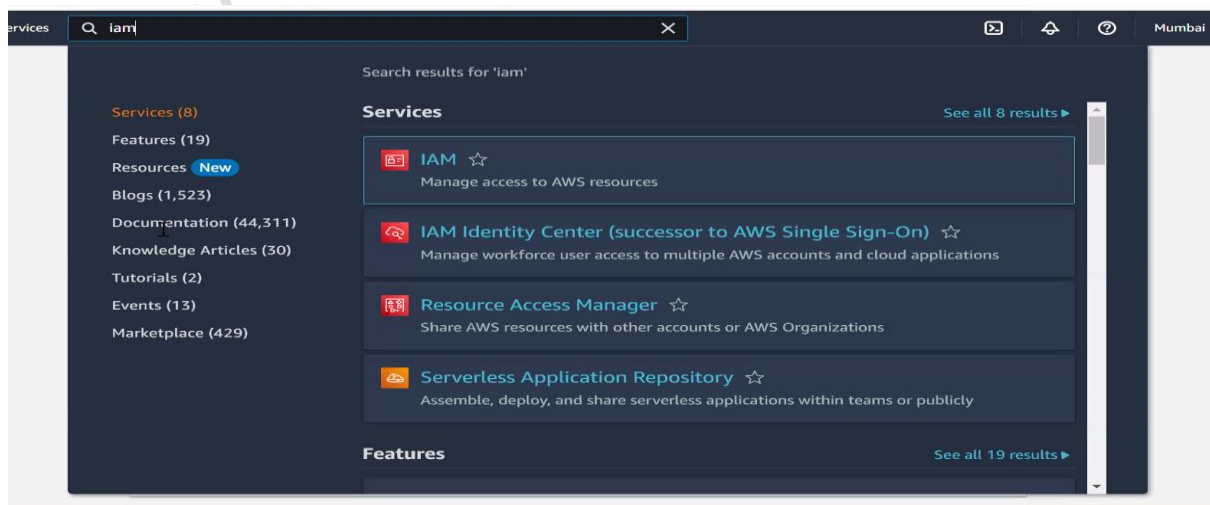
- To use fully managed service, we have **EKS fargate** which is serverless(anything which is fully managed).
- Any computer running and controlling multiple computers is called **control plane**. It manages entire k8s, have full control on entire master computer.
- So, Control plane==Master node
Worker node==Slave node
- For master node AWS give the serverless experience. So, it has pricing as 0.10 per hour. We have to pay for control plane.

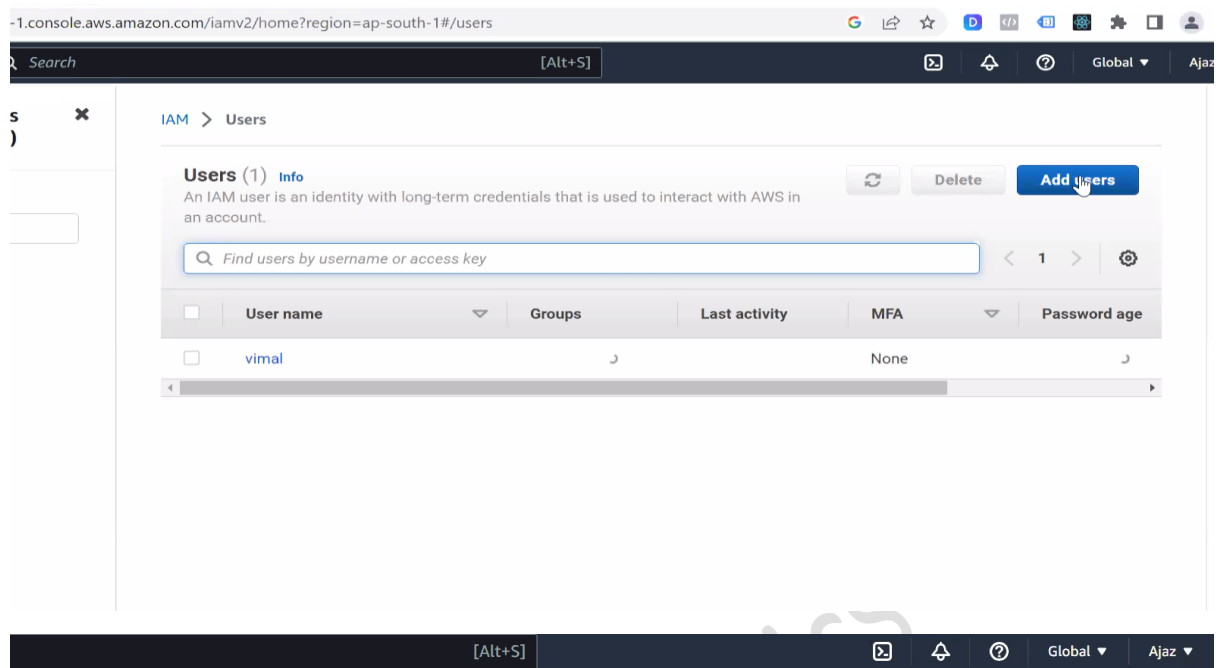
PRACTICAL - Launch EKS Cluster

- 3 ways by which we can use AWS EKS service
 1. **WebUI**
 2. 3rd Party tool → **Terraform**
 3. 3rd Party tool → **eksctl** tool, specifically for EKS service of AWS
This tool was developed by Weaveworks.



Now create user by using **IAM**.





er

Specify user details

User details

User name

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☐ Provide user access to the AWS Management Console - *optional*

If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

i If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)

Cancel

Next

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐

Add user to group

Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐

Copy permissions

Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒

Attach policies directly

Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1050)

Choose one or more policies to attach to your new user.

Filter distributions by text, property or value

< 1 2 3 4 5 6 7 ... 53 >

Create policy

Search

[AWS]

Global

Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

Copy all group memberships, attached managed policies, and inline policies from an existing user.

Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1/1050)

Choose one or more policies to attach to your new user.

Filter distributions by text, property or value

< 1 2 3 4 5 6 7 ... 53 >

☐

Policy name

Type

Attached entities

☐

AccessAnalyzerService...

AWS managed

0

☒

AdministratorAccess

AWS managed - job function

1

☐

AdministratorAccess-A...

AWS managed

0

☐

AdministratorAccess-A...

AWS managed

0

☐

AlexaForBusinessDevic...

AWS managed

0

☐

AlexaForBusinessFullA...

AWS managed

0

© 2023, Amazon Web Services India Private Limited or its affiliates. Privacy Terms

Permissions summary

< 1 >

Name

Type

Used as

AdministratorAccess

AWS managed - job function

Permissions policy

Tags - optional

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel

Previous

Create user

pg. 6

Permissions

Groups

Tags

Security credentials

Access Advisor

Permissions policies (1)

Permissions are defined by policies attached to the user directly or through groups.

Find policies

☐	Policy name ↗	Type
☐	 AdministratorAccess	AWS managed - job function

Create access key

Access key best practices & alternatives

Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.

☒ Command Line Interface (CLI)
You plan to use this access key to enable the AWS CLI to access your AWS account.

☐ Local code
You plan to use this access key to enable application code in a local development environment to access your AWS account.

☐ Application running on an AWS compute service
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

☐ Third-party service
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.

Create access key

Set description tag - optional

The description for this access key will be attached to this user as a tag and shown alongside the access key.

Description tag value

Describe the purpose of this access key and where it will be used. A good description will help you rotate this access key confidently later.

Maximum 256 characters. Allowed characters are letters, numbers, spaces representable in UTF-8, and: _ . : / = + - @

Cancel

Previous

Create access key

Access key and secret key will be created. Now use this access key and secret key to configure AWS.

[AWS]

The screenshot shows the AWS IAM console for a user named 'myuser1'. The 'Security credentials' tab is selected, showing 'Console access' as 'Disabled'. A terminal window is open in the foreground, displaying the following commands and output:

```
C:\Users\Vimal Daga> aws configure
AWS Access Key ID [*****]: AKIAZSC
AWS Secret Access Key [*****]: Ssk
Default region name [ap-southeast-1]: ap-south-1
Default output format [json]:

C:\Users\Vimal Daga> eksctl create cluster --name amazing-creature-1679139187
[!] 1 error(s) occurred and cluster hasn't been created
[!] to cleanup resources, run 'eksctl delete cluster --name amazing-creature-1679139187'
[!] waiting for CloudFormation stack "eksctl-amazing-creature-1679139187" to be ready
Error: failed waiting for successful resource stack
Error: failed to create cluster "amazing-creature-1679139187"
```

After configuring AWS, we will install eksctl

The screenshot shows a Google search for 'eksctl' with approximately 67,300 results. The top result is from 'eksctl.io'. Below the search results, there are 'People also ask' questions. The bottom part of the screenshot shows the 'eksctl.io' website, which is titled 'eksctl - The official CLI for Amazon EKS'. The website features a sidebar with navigation links, a main content area with a 'New for 2023' section, and a footer with a search bar.

Now go to releases → latest → eksctl_windows_amd64.zip

The screenshot shows the GitHub repository page for `weaveworks/eksctl`. The top section displays a list of recent commits with their titles and timestamps. The right sidebar shows repository statistics: 4.4k stars, 120 watching, and 1.3k forks. The main content area highlights the latest release, `eksctl 0.134.0`, with a 'Latest' badge. Below the release title, the 'Assets' section lists 13 downloadable files, including checksums, binaries for various operating systems and architectures, and source code. The bottom of the page shows the Windows taskbar with the search bar and system tray.

Contributors

Contributors: rothgar, Jennas-Lee, and 2 other contributors

Assets 13

Assets:

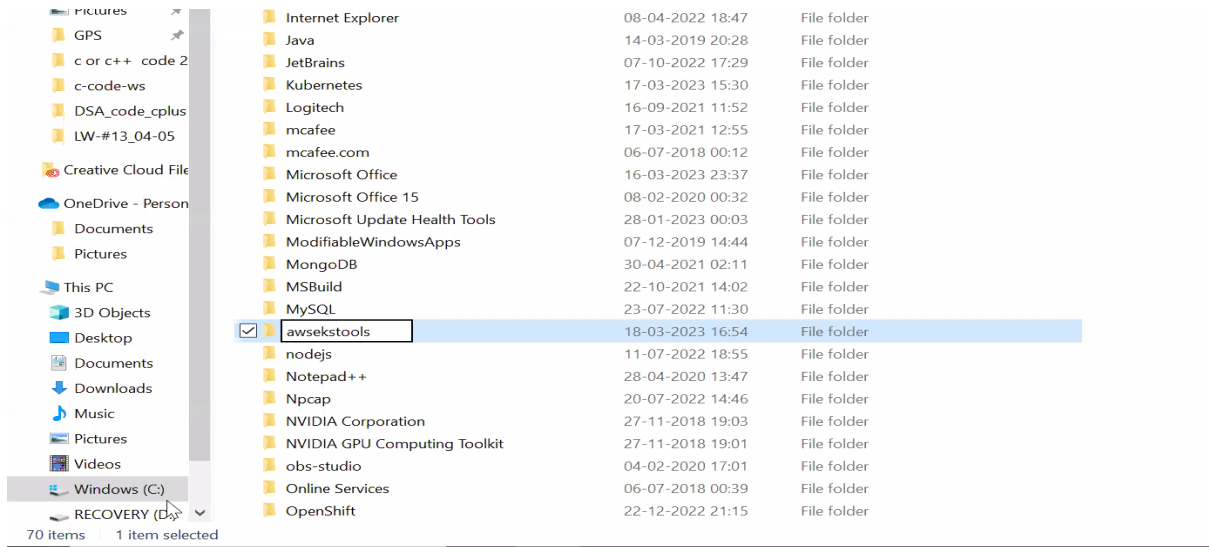
- eksctl_checksums.txt (918 Bytes) yesterday
- eksctl_Darwin_amd64.tar.gz (32.2 MB) yesterday
- eksctl_Darwin_arm64.tar.gz (31.2 MB) yesterday
- eksctl_Linux_amd64.tar.gz (30.7 MB) yesterday
- eksctl_Linux_arm64.tar.gz (27.8 MB) yesterday
- eksctl_Linux_armv6.tar.gz (28.2 MB) yesterday
- eksctl_Linux_armv7.tar.gz (28.2 MB) yesterday
- eksctl_Windows_amd64.zip (30.8 MB) yesterday
- eksctl_Windows_arm64.zip (28 MB) yesterday
- eksctl_Windows_armv6.zip (28.7 MB) yesterday
- Source code (zip) yesterday
- Source code (tar.gz) yesterday

1 person reacted

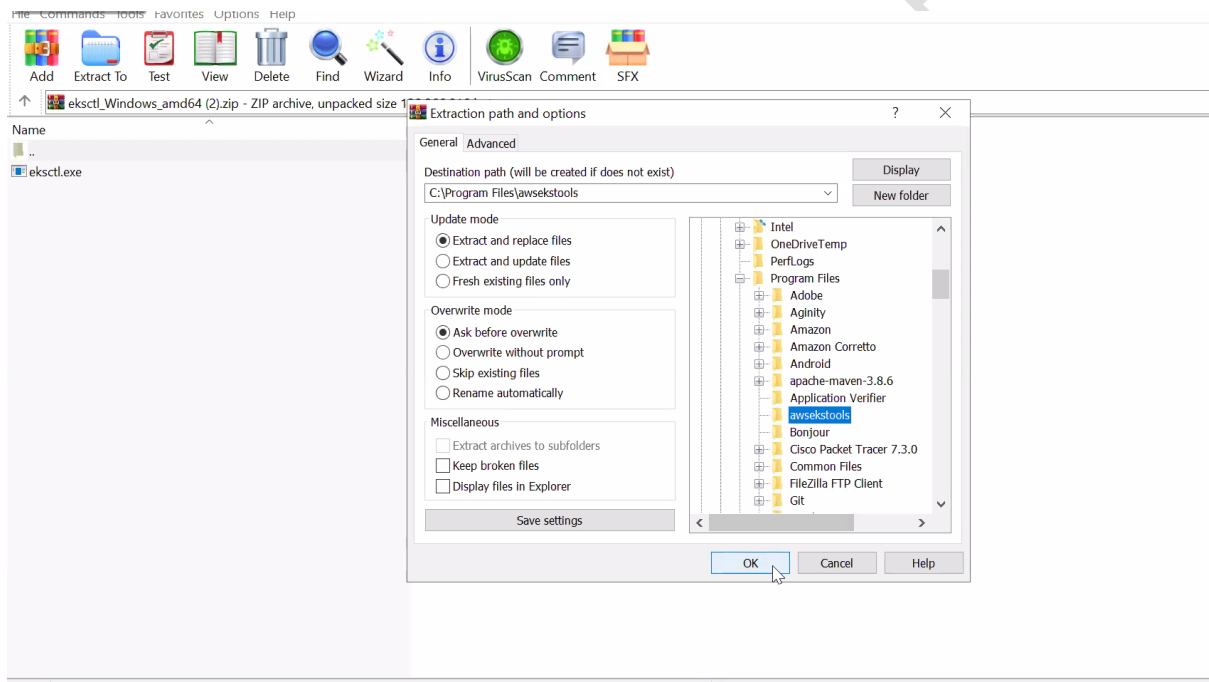
eksctl_Windows_...zip (14.6/30.8 MB, 2 secs left)

After downloading create a folder say `awseksctools` and extract downloaded data into that folder.

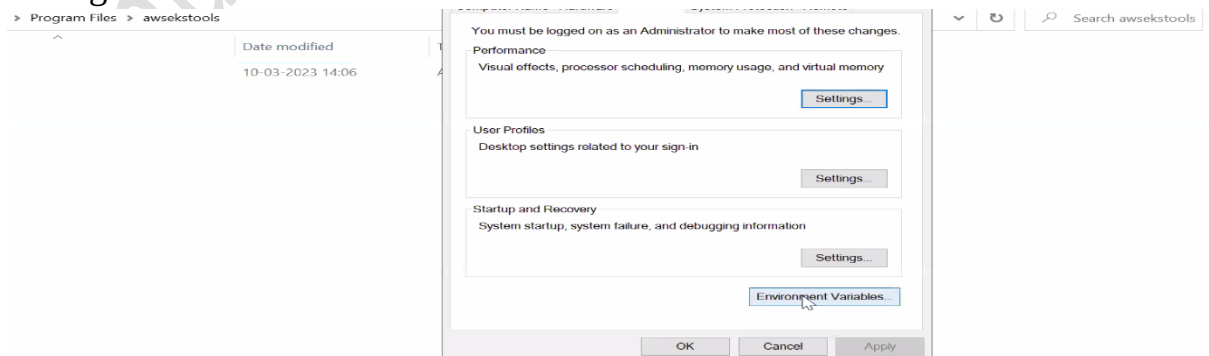
[AWS]

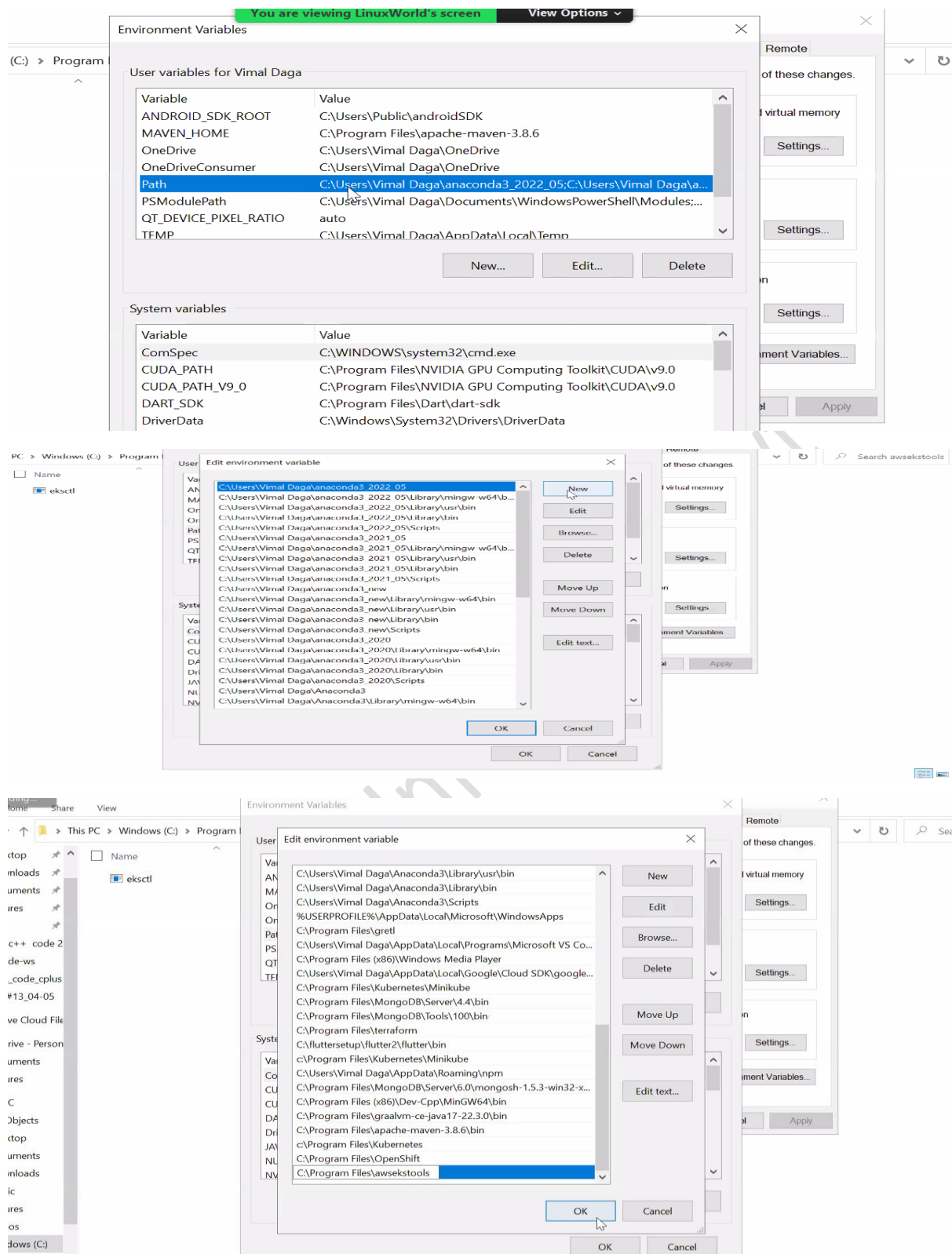


Now extract that into the folder



Now go to environmental variables





After setting environment variable we will create a cluster by using command **eksctl create cluster**.

[AWS]

```
Command Prompt - eksctl.exe create cluster

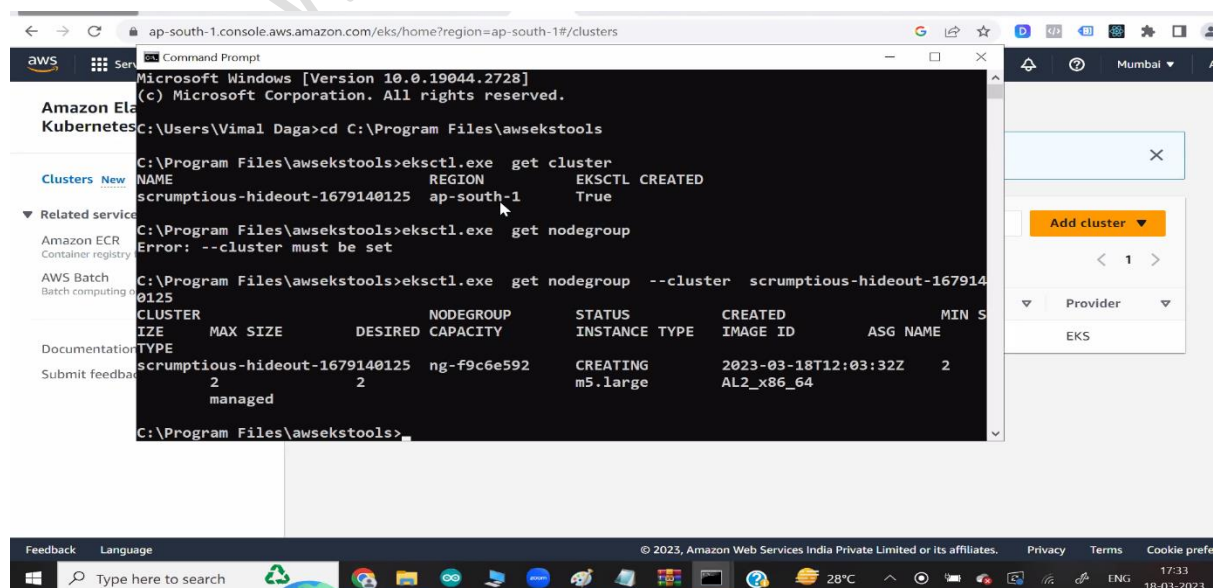
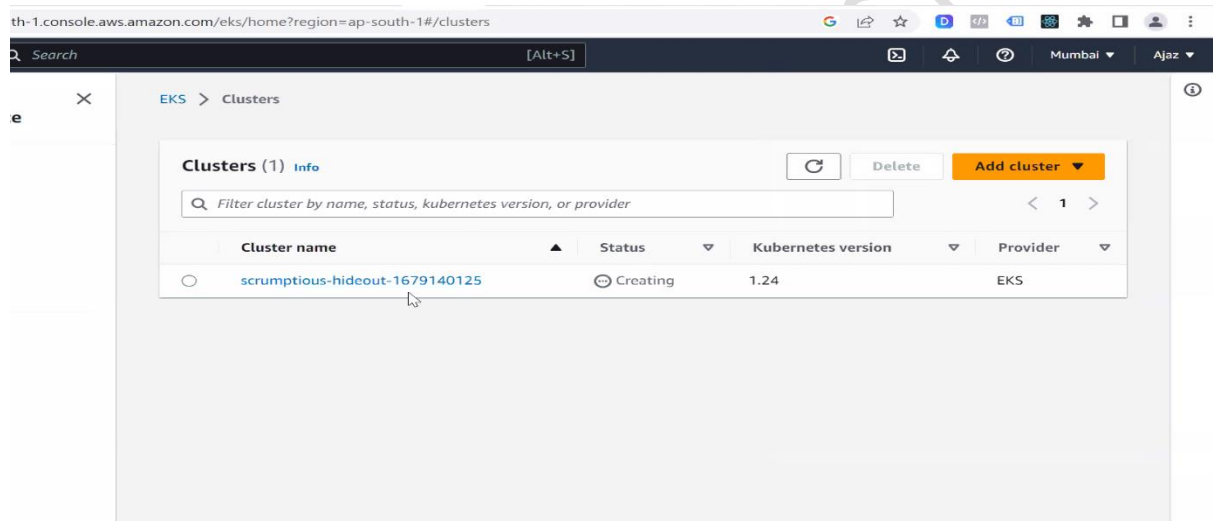
Directory of C:\Program Files\awseksstools
18-03-2023 16:56 <DIR> .
18-03-2023 16:56 <DIR> ..
10-03-2023 14:06 130,966,016 eksctl.exe
1 File(s) 130,966,016 bytes
2 Dir(s) 1,323,294,720 bytes free

C:\Program Files\awseksstools>eksctl.exe version
0.133.0

C:\Program Files\awseksstools>aws configure
AWS Access Key ID [*****OVOM]:
AWS Secret Access Key [*****oBOz]:
Default region name [us-east-1]: ap-south-1
Default output format [json]:

C:\Program Files\awseksstools>eksctl.exe create cluster
2023-03-18 17:18:46 [0] eksctl version 0.133.0
2023-03-18 17:18:46 [0] using region ap-south-1
```

Cluster created successfully.



The top screenshot shows the AWS EKS console for the region 'ap-south-1'. A notification banner states: "New Kubernetes versions are available for 1 cluster." Below this, the 'Clusters (1) Info' section shows a single cluster named 'scrumptious-hideout-1679140125' with a status of 'Active', Kubernetes version '1.24', and provider 'EKS'. A button 'Update now' is visible next to the version.

The bottom screenshot shows a terminal window with the following commands and output:

```
C:\Program Files\awseks\tools>eksctl.exe get cluster
NAME          REGION    EKSCTL CREATED
scrumptious-hideout-1679140125  ap-south-1  True

C:\Program Files\awseks\tools>eksctl.exe get nodegroup
Error: --cluster must be set

C:\Program Files\awseks\tools>eksctl.exe get nodegroup --cluster scrumptious-hideout-1679140125
CLUSTER      NODEGROUP  STATUS    CREATED          MIN SIZE  MAX SIZE  DESIRED CAPACITY
scrumptious-hideout-1679140125  ng-f9c6e592  CREATING  2023-03-18T12:03:32Z  2         2         2
m5.large
managed

C:\Program Files\awseks\tools>eksctl.exe get nodegroup --cluster scrumptious-hideout-1679140125
CLUSTER      NODEGROUP  STATUS    CREATED          MIN SIZE  MAX SIZE  DESIRED CAPACITY
scrumptious-hideout-1679140125  ng-f9c6e592  CREATING  2023-03-18T12:03:32Z  2         2         2
m5.large
managed
```

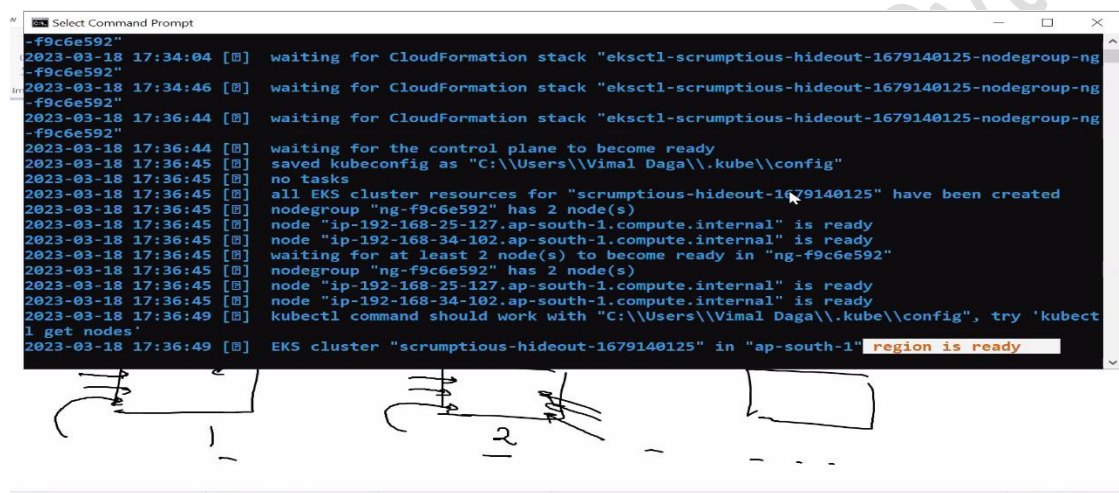
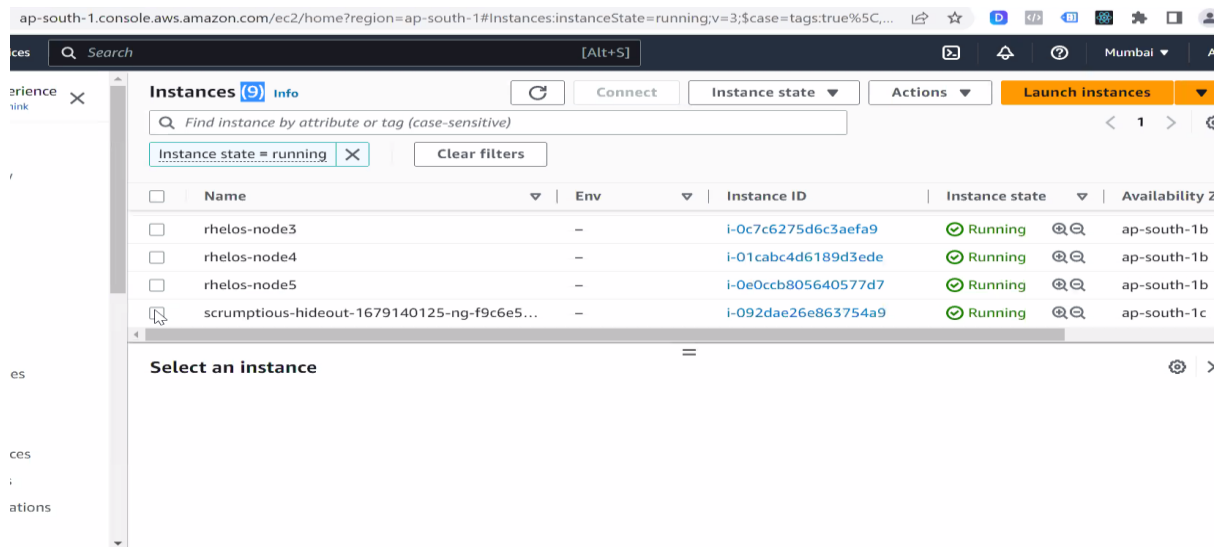
Below the terminal, the console shows a message: "No Nodes. This cluster does not have any Nodes, or you don't have permission to view them."

By default whenever cluster is created EKS gives 2 worker nodes with some instance type.

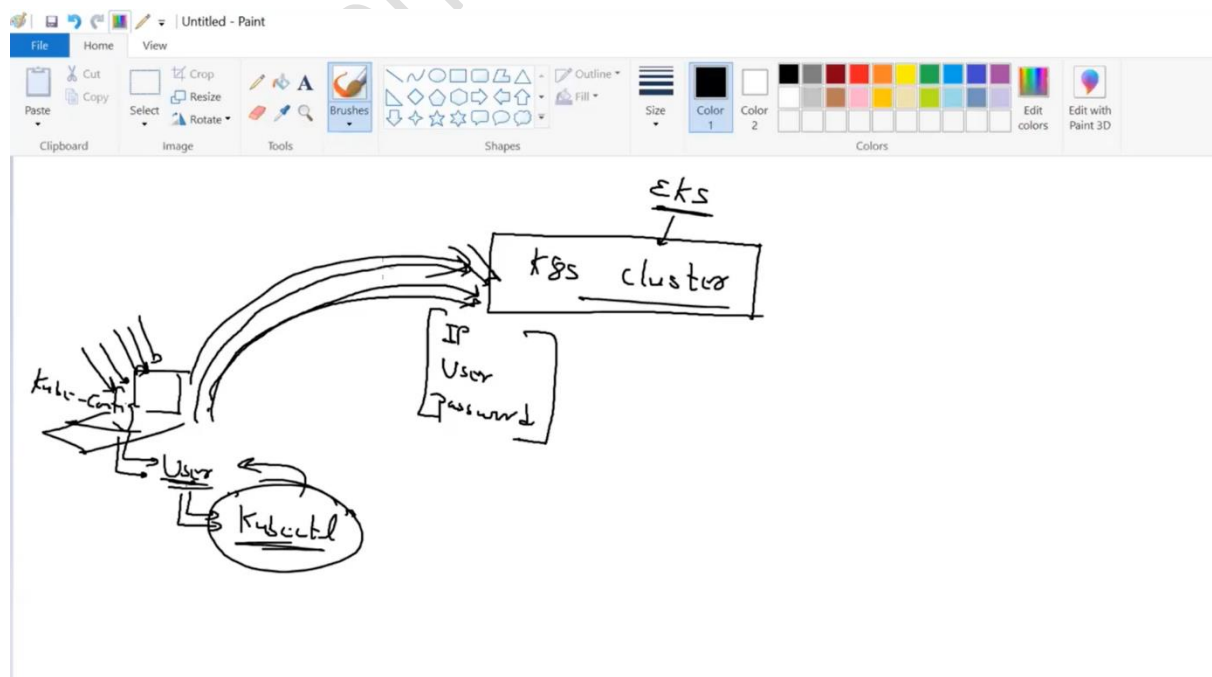
The screenshot shows the AWS EC2 console for the region 'ap-south-1'. The 'Instances' tab is selected, and a filter for 'Instance state = running' is applied. The following table lists the instances:

Name	Env	Instance ID	Instance state	Availability Zone
os2	-	i-071e38919945a49d2	Running	ap-south-1a
scrumptious-hideout-1679140125-n...	-	i-066433daede7c8818	Running	ap-south-1a
rheloc-controller	Dev	i-0c85a7ea85c0c4f8a	Running	ap-south-1b
rheloc-node1	-	i-03a41595ade300dc0	Running	ap-south-1b

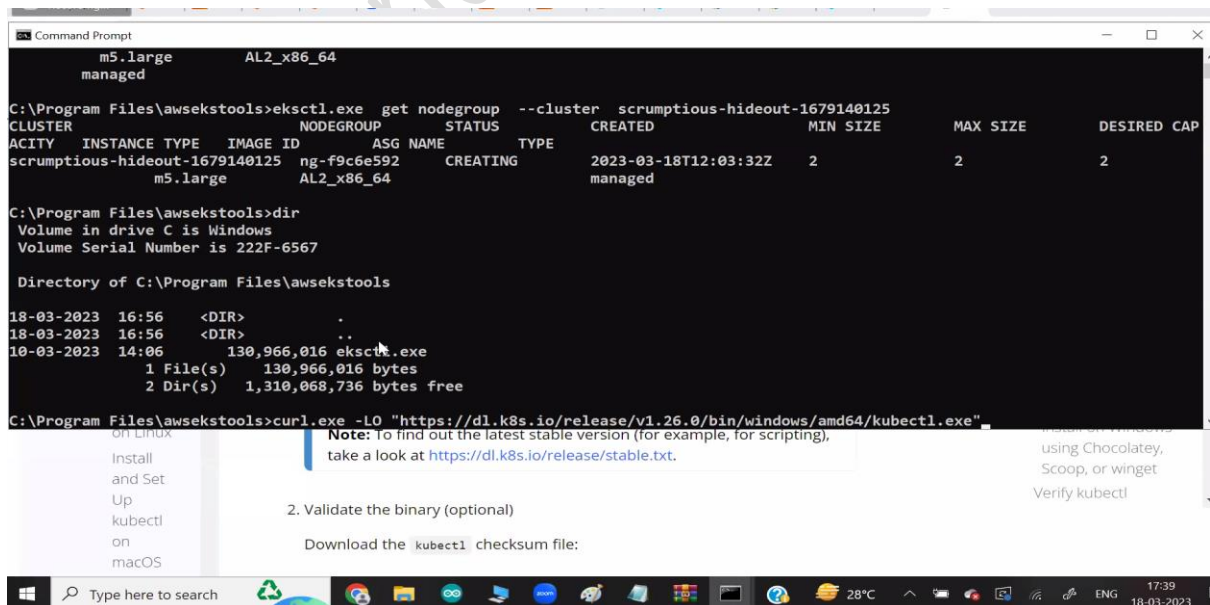
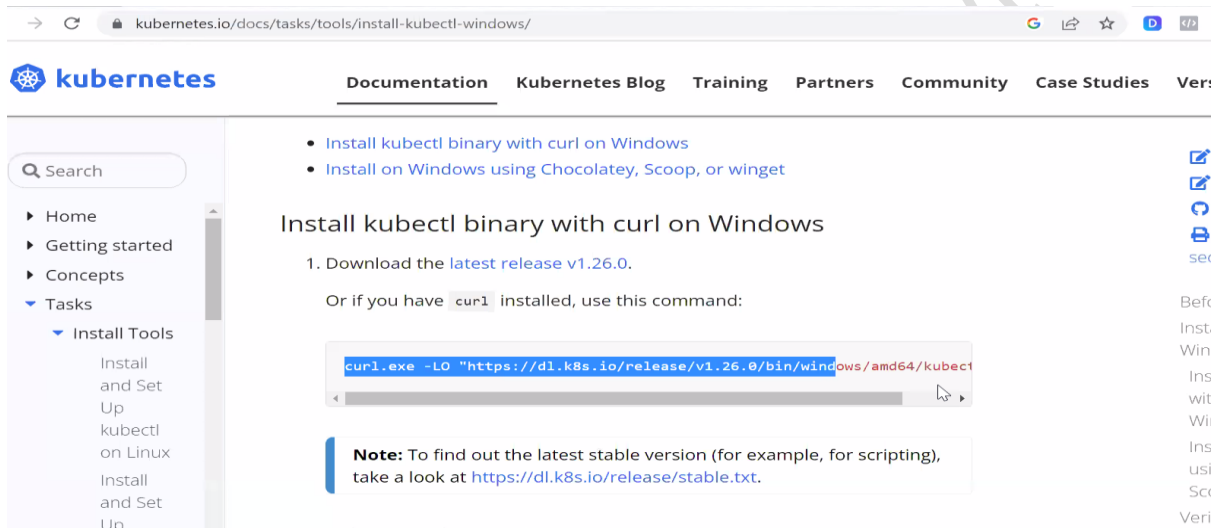
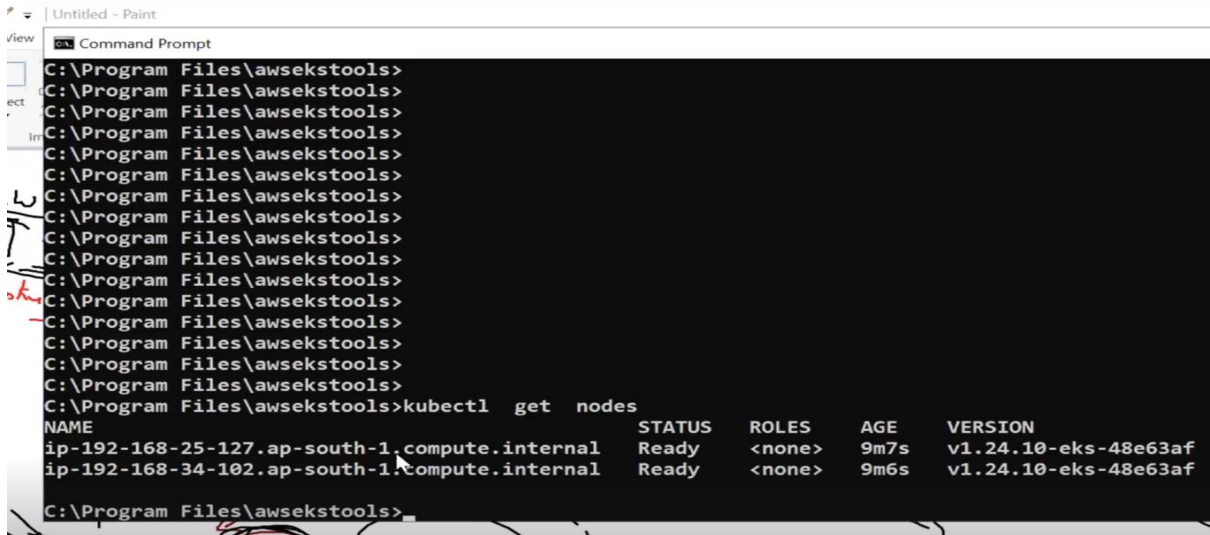
A modal window titled 'Select an instance' is open at the bottom of the console.



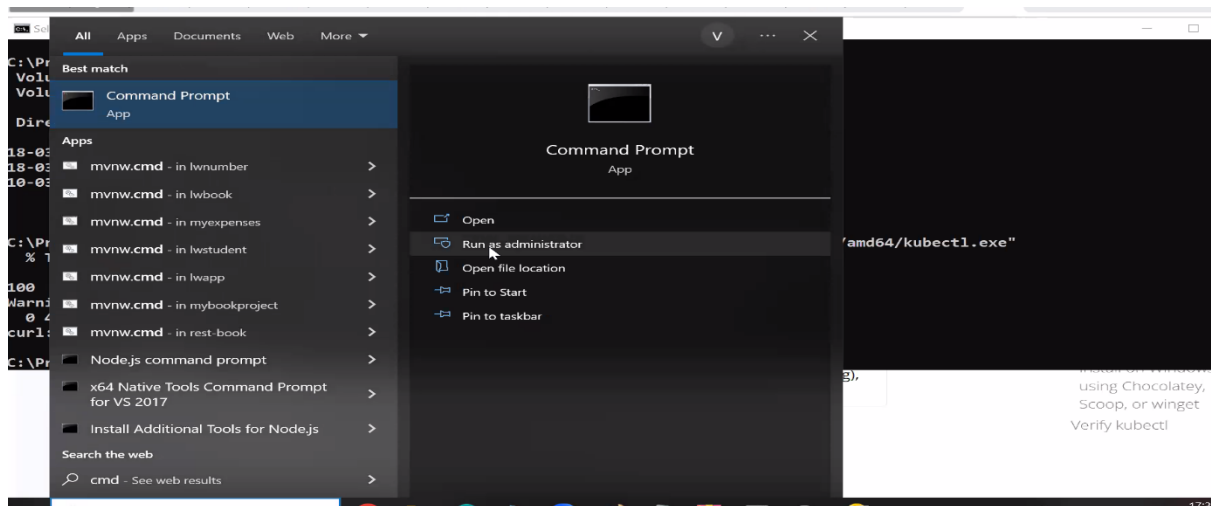
- kube config file contains credentials, it contains kubectl command.



[AWS]



Ng here is the node group or group of nodes.



Best match

- Command Prompt App

Apps

- mvnw.cmd - in lwnumber
- mvnw.cmd - in lwbook
- mvnw.cmd - in myexpenses
- mvnw.cmd - in lwstudent
- mvnw.cmd - in lwapp
- mvnw.cmd - in mybookproject
- mvnw.cmd - in rest-book
- Node.js command prompt
- x64 Native Tools Command Prompt for VS 2017
- Install Additional Tools for Node.js

Search the web

- cmd - See web results

Command Prompt App

- Open
- Run as administrator
- Open file location
- Pin to Start
- Pin to taskbar

amd64/kubectl.exe"

using Chocolatey, Scoop, or winget

Verify kubectl

kubernetes.io/docs/tasks/tools/install-kubectl-windows/

Administrator: Command Prompt - curl.exe -LO "https://dl.k8s.io/release/v1.26.0/bin/windows/amd64/kubectl.exe"

Microsoft Windows [Version 10.0.19044.2728]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>cd C:\Program Files\awseksstools

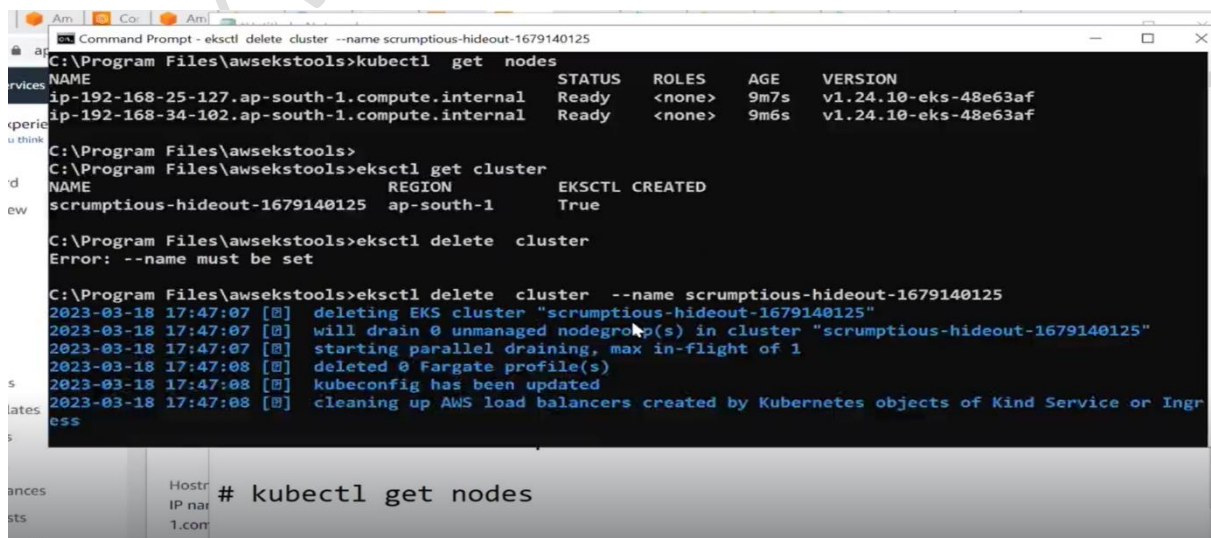
C:\Program Files\awseksstools>curl.exe -LO "https://dl.k8s.io/release/v1.26.0/bin/windows/amd64/kubectl.exe"

% Total	% Received	% Xferd	Average Speed	Time Total	Time Spent	Time Left	Current Speed
0	0	0	0	0	0	--:--:--	0

2. Validate the binary (optional)

Download the `kubectl` checksum file:

Now delete that cluster.



Command Prompt - eksctl delete cluster --name scrumptious-hideout-1679140125

C:\Program Files\awseksstools>kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-25-127.ap-south-1.compute.internal	Ready	<none>	9m7s	v1.24.10-eks-48e63af
ip-192-168-34-102.ap-south-1.compute.internal	Ready	<none>	9m6s	v1.24.10-eks-48e63af

C:\Program Files\awseksstools>eksctl get cluster

NAME	REGION	EKSCTL CREATED
scrumptious-hideout-1679140125	ap-south-1	True

C:\Program Files\awseksstools>eksctl delete cluster

Error: --name must be set

C:\Program Files\awseksstools>eksctl delete cluster --name scrumptious-hideout-1679140125

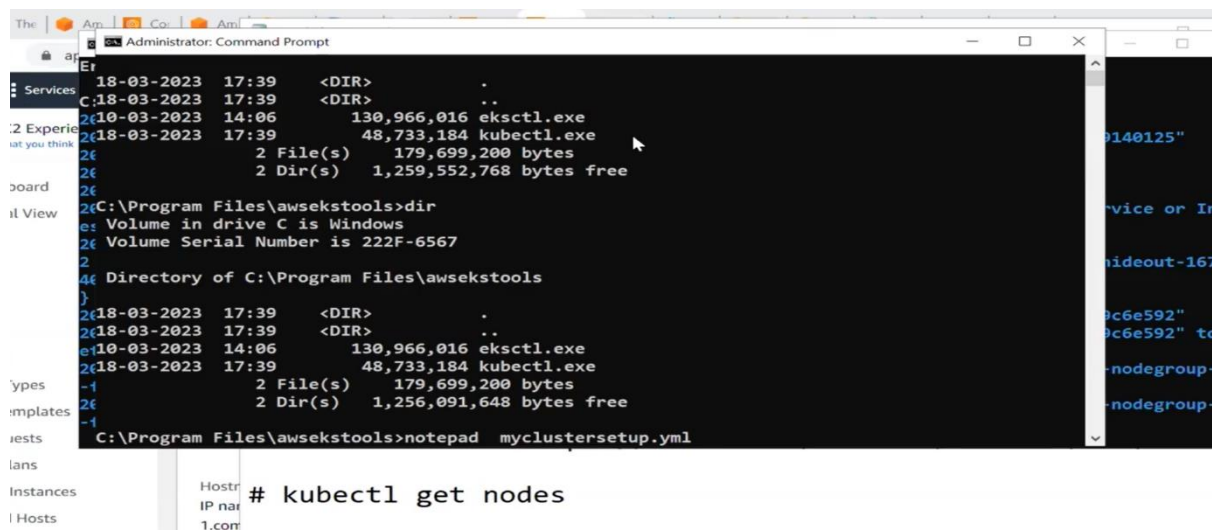
```

2023-03-18 17:47:07 [0] deleting EKS cluster "scrumptious-hideout-1679140125"
2023-03-18 17:47:07 [0] will drain 0 unmanaged nodegroup(s) in cluster "scrumptious-hideout-1679140125"
2023-03-18 17:47:07 [0] starting parallel draining, max in-flight of 1
2023-03-18 17:47:08 [0] deleted 0 Fargate profile(s)
2023-03-18 17:47:08 [0] kubeconfig has been updated
2023-03-18 17:47:08 [0] cleaning up AWS load balancers created by Kubernetes objects of Kind Service or Ingress
  
```

kubectl get nodes

[AWS]

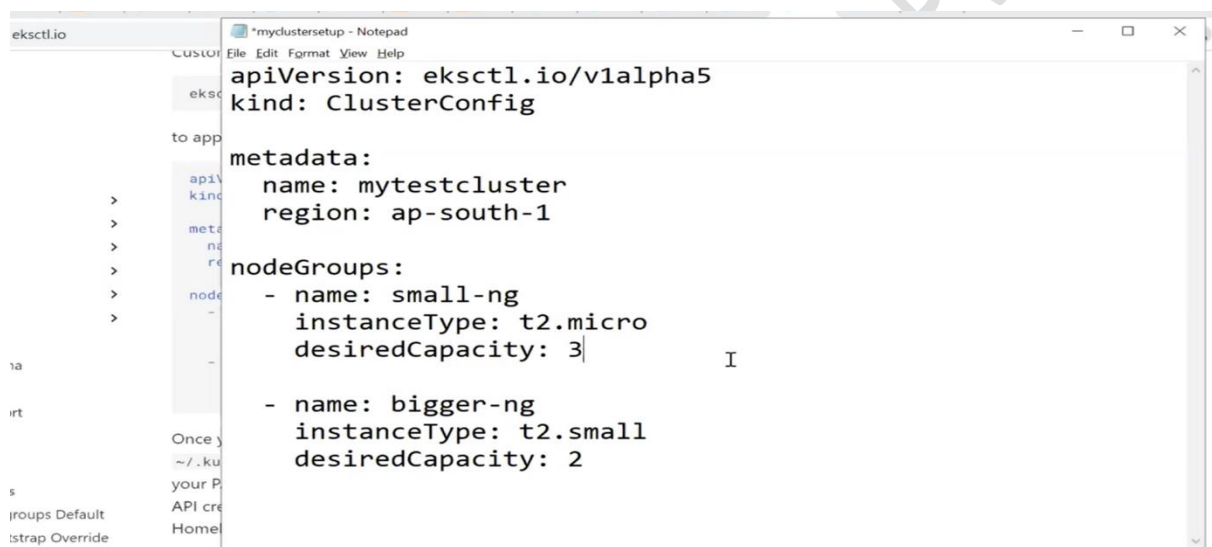
Launching cluster again by yml file.



```
Administrator: Command Prompt
C:\Program Files\awseks tools>dir
18-03-2023 17:39 <DIR> .
18-03-2023 17:39 <DIR> ..
10-03-2023 14:06 130,966,016 eksctl.exe
18-03-2023 17:39 48,733,184 kubectl.exe
2 File(s) 179,699,200 bytes
2 Dir(s) 1,259,552,768 bytes free

C:\Program Files\awseks tools>dir
18-03-2023 17:39 <DIR> .
18-03-2023 17:39 <DIR> ..
10-03-2023 14:06 130,966,016 eksctl.exe
18-03-2023 17:39 48,733,184 kubectl.exe
2 File(s) 179,699,200 bytes
2 Dir(s) 1,259,552,768 bytes free

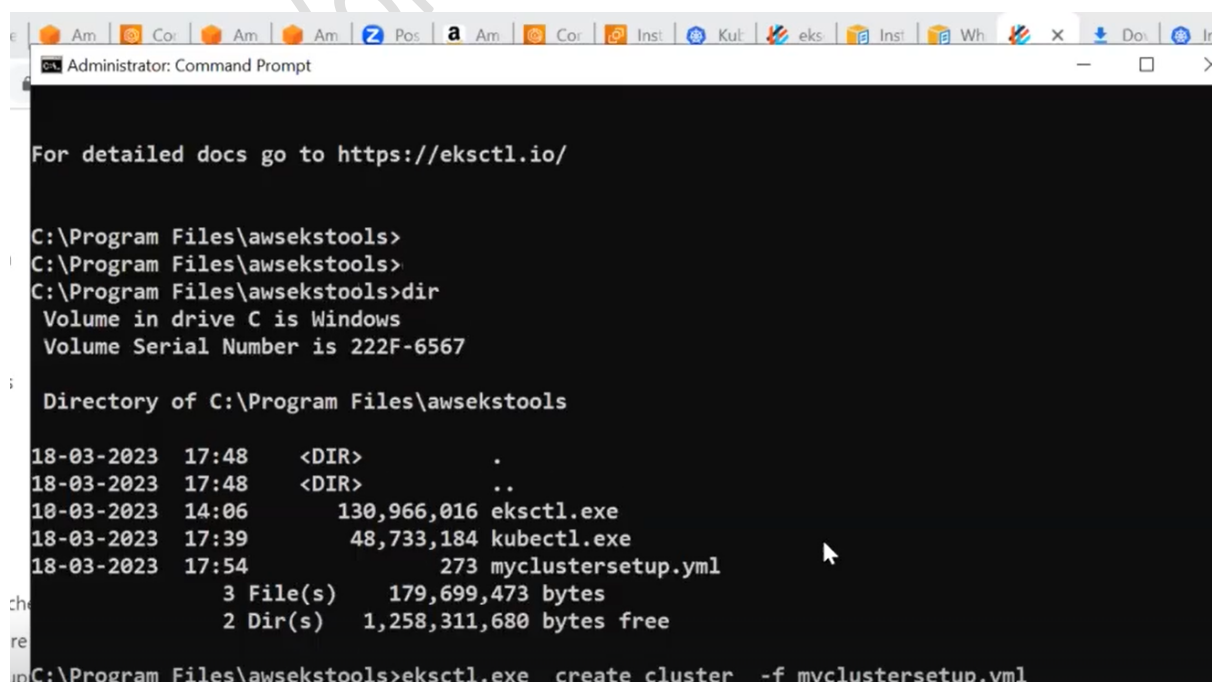
C:\Program Files\awseks tools>notepad myclustersetup.yml
```



```
*myclustersetup - Notepad
File Edit Format View Help
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: mytestcluster
  region: ap-south-1

nodeGroups:
  - name: small-ng
    instanceType: t2.micro
    desiredCapacity: 3
  - name: bigger-ng
    instanceType: t2.small
    desiredCapacity: 2
```



```
Administrator: Command Prompt

For detailed docs go to https://eksctl.io/

C:\Program Files\awseks tools>
C:\Program Files\awseks tools>
C:\Program Files\awseks tools>dir
Volume in drive C is Windows
Volume Serial Number is 222F-6567

Directory of C:\Program Files\awseks tools

18-03-2023 17:48 <DIR> .
18-03-2023 17:48 <DIR> ..
10-03-2023 14:06 130,966,016 eksctl.exe
18-03-2023 17:39 48,733,184 kubectl.exe
18-03-2023 17:54 273 myclustersetup.yml
3 File(s) 179,699,473 bytes
2 Dir(s) 1,258,311,680 bytes free

C:\Program Files\awseks tools>eksctl.exe create cluster -f myclustersetup.yml
```

```

y2023-03-18 17:57:31 [0] all cluster resources were deleted
2
sC:\Program Files\awseksctl\tools>
C:\Program Files\awseksctl\tools>
C:\Program Files\awseksctl\tools>
C:\Program Files\awseksctl\tools>
C:\Program Files\awseksctl\tools>
C:\Program Files\awseksctl\tools>
C:\Program Files\awseksctl\tools>
C:\Program Files\awseksctl\tools>
C:\Program Files\awseksctl\tools>
C:\Program Files\awseksctl\tools>
C:\Program Files\awseksctl\tools>eksctl.exe get cluster
2NAME                REGION    EKSCTL CREATED
2mytestcluster       ap-south-1  True
2

```

Cluster created successfully.

- Whenever we wanted to do something on EKS we use eksctl command and to do something on cluster we use kubectl.

```

*Untitled - Notepad
File Edit Format View Help
1. aws cli
https://docs.aws.amazon.com/cli/latest/userguide/getting-started
# aws --version

2 IAM user: key

3. laptop aws cli to auth aws cloud
# aws configure

4. eksctl to eks cluster
# https://github.com/weaveworks/eksctl/releases
# eksctl version

4. eksctl to eks cluster
# https://github.com/weaveworks/eksctl/releases
# eksctl version

# eksctl create cluster

-----

user/client K8s:

# curl.exe -LO "https://dl.k8s.io/release/v1.26.0/bin/windows

```